Function Types

Functions are objects – store them, pass them, return them. Know the kinds you will meet.

Kinds of functions

Kind	What it is	Example
Built-in	ships with Python	`len`, `print`, `sorted`
User-defined	created with `def`	`def area(w, h): ...`
Lambda	anonymous, one expression	`lambda x: x + 1`
Nested	`def` inside a function	private helper
Closure	inner function that remembers outer variables	`make_counter()`
Recursive	calls itself (needs a base case)	`fact(n - 1)`
Higher-order	takes or returns functions	`map`, `sorted(key=)`, decorators
Generator	uses `yield`; produces values lazily	`def count(): yield 1`
Coroutine	`async def`, awaited	asyncio code
Method	function bound to an object	`"hi".upper()`

First-class objects

```
def shout(s): return s.upper() + "!"
def whisper(s): return s.lower() + "..."

# store in a dict
actions = {"s": shout, "w": whisper}
print(actions["s"]("hey"))

# pass as an argument
def apply(fn, value):
    return fn(value)

print(apply(whisper, "HEY"))
print(type(shout).__name__, type(len).__name__)
```

OUTPUT

```
HEY!
hey...
function builtin_function_or_method
```

Higher-order: returning a function

```
def power(n):
    def raise_to(x):        # inner function / closure
        return x ** n
    return raise_to

cube = power(3)
print(cube(2), callable(cube))
```

OUTPUT

```
8 True
```

Recursive functions

```
def fact(n):
    # base case stops the recursion
    if n <= 1:
        return 1
    return n * fact(n - 1)   # recursive case

print(fact(5))
```

OUTPUT

```
120
```

Python caps recursion at roughly 1000 frames (`sys.getrecursionlimit()`); convert deep recursion to loops.

Method types inside classes

```
class Temp:
    @classmethod
    def make(cls):        # receives the class
        return "class"
    @staticmethod
    def helper():         # receives nothing
        return "static"
    def show(self):       # receives the instance
        return "instance"

t = Temp()
print(t.show(), Temp.make(), Temp.helper())
```

OUTPUT

```
instance class static
```

Type hints for function types

```
from typing import Callable

def apply_twice(fn: Callable[[int], int],
                x: int) -> int:
    return fn(fn(x))

print(apply_twice(lambda n: n + 3, 10))
```

OUTPUT

```
16
```

TIP Handy checks: `callable(obj)`, `type(f)`, `f.__name__`, `inspect.isfunction(f)`.

REMEMBER `f` is the function object; `f()` **calls** it. Passing `f` (no parentheses) is how you hand a function to another function.